Efficient Mamba-Based Accelerator Architecture for Massive-MIMO Indoor Localization Using the LuViRA Dataset

Shengjie Chen
sh5704ch-s@student.lu.se

Chenxi Zhang
ch7210zh-s@student.lu.se

Department of Electrical and Information Technology
Lund University

Supervisor: Ilayda Yaman
Sijia Cheng

Examiner: Pietro Andreani

May 15, 2026

Printed in Sweden
Tryckeriet i E-huset, Lund

Abstract

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Popular Science Summary

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

Fusce mauris. Vestibulum luctus nibh at lectus. Sed bibendum, nulla a faucibus semper, leo velit ultricies tellus, ac venenatis arcu wisi vel nisl. Vestibulum diam. Aliquam pellentesque, augue quis sagittis posuere, turpis lacus congue quam, in hendrerit risus eros eget felis. Maecenas eget erat in sapien mattis porttitor. Vestibulum porttitor. Nulla facilisi. Sed a turpis eu lacus commodo facilisis. Morbi fringilla, wisi in dignissim interdum, justo lectus sagittis dui, et vehicula libero dui cursus dui. Mauris tempor ligula sed lacus. Duis cursus enim ut augue. Cras ac magna. Cras nulla. Nulla egestas. Curabitur a leo. Quisque egestas wisi eget nunc. Nam feugiat lacus vel est. Curabitur consectetuer.

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Sed commodo posuere pede. Mauris ut est. Ut quis purus. Sed ac odio. Sed

vehicula hendrerit sem. Duis non odio. Morbi ut dui. Sed accumsan risus eget odio. In hac habitasse platea dictumst. Pellentesque non elit. Fusce sed justo eu urna porta tincidunt. Mauris felis odio, sollicitudin sed, volutpat a, ornare ac, erat. Morbi quis dolor. Donec pellentesque, erat ac sagittis semper, nunc dui lobortis purus, quis congue purus metus ultricies tellus. Proin et quam. Class aptent taciti sociosqu ad litora torquent per conubia nostra, per inceptos hymenaeos. Praesent sapien turpis, fermentum vel, eleifend faucibus, vehicula eu, lacus.

Table of Contents

List of Figures

List of Tables

Chapter 1
Introduction

1.1 Background

Indoor localization is becoming an important capability for future 6G wireless systems, since 6G is expected to support not only ubiquitous communication but also high-accuracy localization and high-resolution sensing services [1]. Localization is also regarded as a key enabler for future 6G wireless systems [2]. With the development of location-aware applications such as robotic navigation, emergency healthcare, and smart transportation, wireless localization is expected to support a wider range of location-aware services [4]. At the same time, real-time indoor positioning has long remained a challenging problem [3]. In indoor wireless localization systems, localization performance is affected by environmental factors such as multipath and non-line-of-sight propagation [6].In this context, Massive MIMO is particularly promising for high-precision indoor localization because large antenna arrays can provide higher angular resolution and support excellent localization accuracy even under limited bandwidth [5].

From an algorithmic perspective, traditional localization methods include proximity, triangulation or trilateration, fingerprint matching, and SLAM [4]. However, conventional signal-processing-based methods often suffer from high algorithmic complexity and may require base-station array calibration [4]. Machine learning provides an attractive alternative, since wireless localization can be formulated as a regression task from channel state information to user positions [5]. ML models can exploit raw transfer functions, received signal strength, power delay profiles, angular spectra, and correlation functions as input channel features [4]. Measurement-based studies using the Lund University Massive MIMO testbed show that spatial covariance matrices can capture angular-domain information, while channel impulse responses can capture delay-domain information [5]. By combining angular-domain and delay-domain information, the proposed ML-based pipeline has achieved centimeter-level localization accuracy in an indoor measurement scenario [5]. However, as the number of antennas increases, the size of neural networks increases significantly, which adds complexity to training, deployment, and inference [4]. Therefore, efficient neural architectures and hardware accelerators are needed for practical Massive-MIMO-based indoor localization systems.

1.2 Related Work

In recent years, machine learning has been investigated for Massive-MIMO-based indoor localization. Tian et al. proposed FCNN-based localization methods that use spatial covariance matrices and truncated channel impulse responses to capture angular- and delay-domain information, achieving centimeter-level accuracy in indoor Massive MIMO measurements [5]. Their later work further introduced an ML-based localization pipeline with multiple processing chains and ensemble learning to fuse different channel fingerprints [4]. However, these studies also show that neural network size increases significantly with the number of antennas in Massive MIMO systems, making training and fine-tuning more challenging [4]. Although subarray-based processing can reduce network size, efficient and hardware-friendly model design remains an important issue for practical deployment [4].

However, most existing ML-based Massive MIMO indoor localization methods rely on FCNNs, CNNs, KNN, or other conventional machine-learning models to process high-dimensional channel fingerprints [4]. These methods can effectively exploit channel features, but for densely connected models such as FCNNs, higher-dimensional inputs usually lead to larger network sizes, more parameters, and higher training and inference cost. Existing work also points out that, when ML-based localization methods are applied to Massive MIMO systems, the increase in network size can make training and fine-tuning more challenging [4]. Therefore, reducing model size, computational complexity, and hardware deployment cost while maintaining localization accuracy remains an important research problem in Massive-MIMO-based indoor localization.

1.3 Goals and Challenges

The goal of this thesis is to investigate the efficiency of Mamba-like architectures for Massive-MIMO indoor localization and further explore their FPGA-based hardware acceleration. Mamba is a selective state-space sequence model that uses input-dependent state updates to selectively propagate or forget information along the sequence dimension, while maintaining linear scaling with sequence length [7]. This property makes Mamba attractive for continuous wireless measurement sequences, because it uses recurrent state updates without the parameter explosion commonly associated with FCNNs when the input dimension increases.

Compared with previous FCNN-based methods that mainly operate on single snapshots or fixed fingerprints, Mamba can process short sequences of observations jointly and exploit temporal dynamics in the wireless channel. Therefore, this thesis aims to evaluate Mamba-family models for LuViRA indoor localization, design a hardware-friendly slim Mamba-like architecture, and implement the key Mamba datapath on FPGA with quantization-aware considerations. The main challenges include preserving localization accuracy after model simplification and quantization, handling the memory and computation pressure caused by high-dimensional CSI, and balancing latency, power, resource utilization, and localization accuracy in the final accelerator design. Recent studies have explored Mamba acceleration in other application scenarios, including FPGA-based quantization and hardware

co-design with computation reordering and fine-grained tiling, pipelined vector processing units, and hybrid dataflows with state-space buffers [9, 10, 11]. Motivated by these works, this thesis further investigates how the Slim Mamba datapath can be reused for indoor localization, with particular attention to datapath reuse, memory access organization, and pipeline granularity.

Chapter 2
Algorithm

2.1 Mamba Basics

Mamba is a sequence modeling architecture based on selective state-space models (selective SSMs). A state-space model represents historical information using a hidden state and describes the dynamics of an input sequence through state transitions. A continuous-time state-space model can be written as

$$\dot{h}(t) = Ah(t) + Bx(t), \tag{2.1}$$

$$y(t) = Ch(t) + Dx(t), \tag{2.2}$$

where $x(t)$ is the input, $h(t)$ is the hidden state, and $y(t)$ is the output. The matrix A controls the state dynamics, B determines how the input affects the state, C maps the hidden state to the output, and D represents a direct input-to-output path. To process discrete sequences, the continuous system can be discretized into a recurrent form:

$$h_t = \bar{A}_t h_{t-1} + \bar{B}_t x_t, \tag{2.3}$$

$$y_t = C_t h_t + Dx_t, \tag{2.4}$$

where x_t is the input at time step t, h_t is the current hidden state, and y_t is the current output. The discretized parameters can be written as

$$\bar{A}_t = \exp(\Delta_t A), \tag{2.5}$$

$$\bar{B}_t = (\Delta_t A)^{-1} \left(\exp(\Delta_t A) - I\right) \Delta_t B_t. \tag{2.6}$$

The key idea of Mamba is to make part of the SSM parameters input-dependent. For example, Δ_t, B_t, and C_t can be generated from the current input x_t through lightweight linear projections. In this way, the model can dynamically decide what information should be preserved, updated, or forgotten according to the current input. Gu and Dao show that making SSM parameters functions of the input allows the model to selectively propagate or forget information along the sequence dimension, while Mamba maintains linear scaling with sequence length [7].

From an architectural perspective, a Mamba block usually consists of an input projection, local feature mixing, a selective scan, a gating branch, and an output projection. The input is first projected and split into a main branch and a gate

branch. The main branch generates the dynamic SSM parameters and updates the hidden state through the selective scan. The gate branch is usually passed through a nonlinear activation function such as SiLU and then multiplied with the SSM output, followed by an output projection. Since the state update can be implemented recurrently over time, Mamba can model contextual information with low sequence complexity and avoids the quadratic complexity commonly associated with attention-based methods on long sequences [7].

Mamba is worth investigating for indoor localization because indoor wireless measurements can often be organized as user trajectories or consecutive channel snapshots, where neighboring snapshots may contain short-term contextual information useful for position estimation. Compared with FCNN-based methods that mainly operate on single snapshots or fixed channel fingerprints, Mamba can progressively aggregate historical observations through its recurrent state and adaptively update the state according to the current input. Therefore, Mamba provides a model candidate that can exploit temporal context while maintaining linear sequence complexity, making it suitable for further investigation in terms of accuracy, efficiency, and hardware implementation for Massive-MIMO-based indoor localization.

2.2 Slim-Mamba

In this work, Slim Mamba is designed as a lightweight Mamba variant for Massive-MIMO-based indoor localization. The model is not obtained by directly adopting the original Mamba architecture, but rather through a task-driven simplification process based on preliminary model exploration. The initial design followed a channel-aware Mamba direction, where multi-branch structures were used to fuse different types of radio features, such as the real and imaginary parts of the channel impulse response (CIR), power features, and first-order difference features. A similar motivation can be found in CMamba, where vanilla Mamba is considered insufficient for modeling cross-channel dependencies in multivariate time series, and additional channel-correlation modeling is introduced to enhance interactions among different channels [8]. For the LuViRA indoor localization task, different radio-domain features may also contain complementary location-related information, making channel-fusion design a reasonable initial direction.

However, preliminary experiments showed that the additional channel-fusion branches provided only limited accuracy improvement for the current localization task, while significantly increasing model size, training time, and hardware mapping complexity. At the same time, the vanilla Mamba v1 baseline, implemented following the original Mamba architecture proposed by Gu and Dao [7], showed a clear generalization gap on LuViRA, whereas short temporal windows were already sufficient to provide useful localization information. Therefore, the final model follows a task-driven simplification principle: it preserves the recurrent state update and input-dependent modulation mechanisms that are most relevant to the localization task, while removing complex multi-branch fusion structures and the full selective-scan parameterization.

Table 2.1 summarizes the measured comparison among the FCNN baseline,

channel mamba, vanilla Mamba v1, and slim mamba on the LuViRA dataset. Compared with channel mamba, slim mamba achieves a comparable test mean error while reducing the model size from 40.4 MB to 3.40 MB and the training time from 73 s to 13 s per epoch. Compared with vanilla Mamba v1, slim mamba significantly improves localization accuracy while maintaining a similarly compact model size. These results indicate that slim mamba provides a better trade-off among localization accuracy, model complexity, training efficiency, and hardware cost for the target indoor localization task.

Table 2.1: Comparison of results on different models.

Model	Input Features	Size (MB)	Time (s/epoch)	it/s	Err. (cm)	HW Cost Proxy
FCNN	CIR	440	N/A	N/A	∼13	High
channel mamba	CIR + Power	40.4	73	13.18	12.9	High
Vanilla Mamba v1	CIR + Power	3.38	33	32.62	24.74	Medium
slim mamba	CIR + Power	3.40	13	75.30	13.5	Low

As illustrated in Fig. 2.1, the Slim Mamba block first applies normalization to the input tokens, followed by a linear input projection that maps the input into an internal feature space. The projected representation is then split into two paths: a state input branch and a gate branch. The state input branch is used for state-space model (SSM) state updating, while the gate branch is used for output modulation.

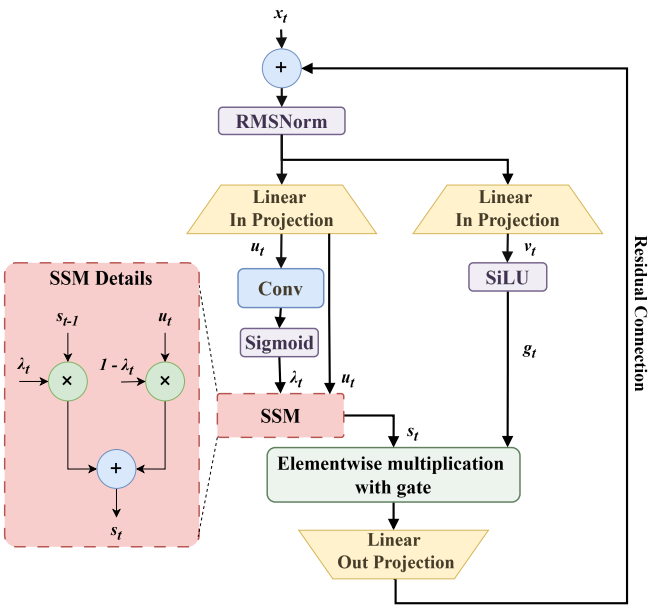

Figure 2.1: Simplified datapath of the Slim Mamba block.

In the state input branch, the update gate generator produces an input-

dependent update coefficient λ_t from the current state input:

$$\lambda_t = \sigma(W_\lambda * u_t + b_\lambda), \tag{2.7}$$

where $*$ denotes the convolution operation, $\sigma(\cdot)$ is the sigmoid function, and λ_t is constrained to the range $(0, 1)$ so that it can control the interpolation between the previous state and the current input. The recurrent state update is computed as

$$s_t = \lambda_t \odot s_{t-1} + (1 - \lambda_t) \odot u_t, \tag{2.8}$$

where s_{t-1} is the previous state, u_t is the current state input, and λ_t is the update gate that controls the balance between the historical state and the current input. When λ_t is fixed, this recurrence is equivalent to an exponential moving average, since older inputs receive weights that decay as powers of λ. In Slim Mamba, however, λ_t is dynamically generated from the current input, and the module is therefore more accurately described as an input-dependent gated state update. Compared with the full (A, B, C, D) selective-scan parameterization and low-rank discretization in the original Mamba, this formulation leads to a more regular computation path and is more suitable for hardware implementation [7].

In parallel, the gate branch is passed through a SiLU activation to generate the output gate:

$$g_t = \mathrm{SiLU}(v_t). \tag{2.9}$$

The output gate then modulates the updated state through element-wise multiplication:

$$z_t = s_t \odot g_t. \tag{2.10}$$

This gating operation allows the model to regulate the effective information from the SSM output according to the current input. The result is then passed through a linear output projection and added back to the input tokens through a residual connection. The residual path helps preserve the original input information and improves training stability. Overall, Slim Mamba retains the recurrent state update and gated modulation mechanisms of Mamba, while removing complex channel-fusion branches that provide limited benefit for the current task.

Compared with channel mamba, Slim Mamba adopts a single-path backbone and reduces branch control and cross-channel fusion overhead. Compared with vanilla Mamba, it simplifies the state-update parameterization and concentrates the main computation on regular operators such as linear projection, update gate generation, state update, element-wise multiplication, and output projection. This structure is more suitable for pipelined FPGA implementation and fixed-point quantization, and provides a clear datapath foundation for the subsequent Mamba-based indoor localization accelerator.

Chapter 3
Method

3.1 Hardware Architecture

This section presents the overall hardware architecture of the Slim-Mamba FPGA accelerator. The design is deployed on a Zynq MPSoC platform, where the processing system (PS) handles software control and DDR management, while the programmable logic (PL) implements the accelerator datapath. The PS-side DDR4 is used as off-chip runtime storage for input and output buffers. The input vector x_t is read from DDR through the memory-mapped to stream (MM2S) channel of the AXI DMA and sent to the PL as an AXI-Stream. The output y_t is written back to DDR through the stream to memory-mapped (S2MM) channel. The PS configures the accelerator through AXI-Lite control/status registers, including input preload, computation start, status polling, and interrupt control.

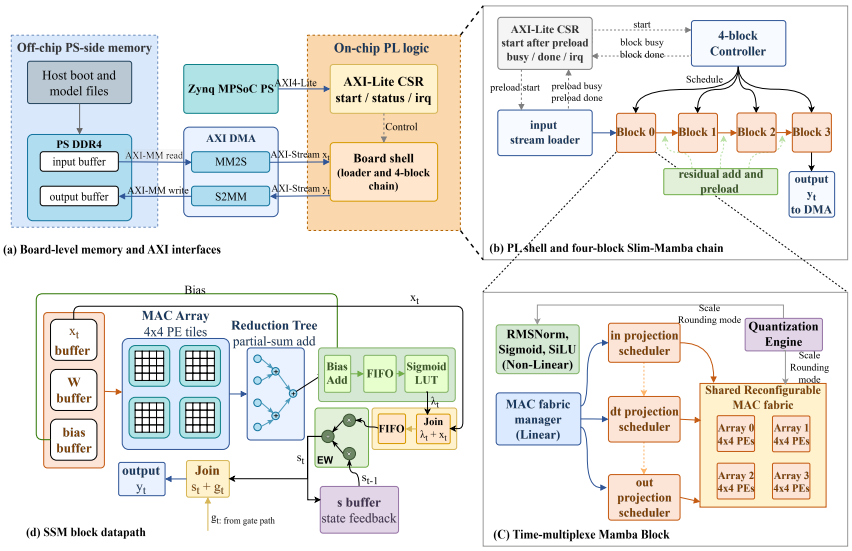

Figure 3.1: Diagram of Slim-Mamba overall hardware architecture.

Inside the PL, the board shell connects the external AXI interfaces to the compute pipeline formed by four Slim-Mamba blocks. The AXI-Lite CSR first starts the input stream loader, which writes the incoming AXI-Stream data into the on-chip input buffer of the first block. After the preload stage is completed, the shell allows the 4-block controller to start computation. The controller schedules the four blocks sequentially. Between adjacent blocks, the output of the current block is added to the residual using saturated fixed-point arithmetic and then preloaded into the input buffer of the next block. The final block output is not passed through another block-level residual preload stage, but is directly streamed to the DMA as y_t.

Within each Slim-Mamba block, the main MAC-intensive stages share a reconfigurable MAC fabric. The input projection scheduler, the Δ projection scheduler, and the output projection scheduler generate tiled operands from on-chip weight and activation buffers. The MAC fabric manager selects the active scheduler and sends its operands to the shared $4 \times 4 \times 4$ MAC fabric. This fabric consists of four 4×4 PE arrays. By time-multiplexing the same fabric across different projection stages, the design avoids instantiating separate MAC arrays and reduces DSP and on-chip interconnect cost.

The design also uses a unified reconfigurable quantization stage. After MAC or element-wise computation, intermediate results are scaled, rounded, and saturated according to the current scheduler and datapath mode, and are then mapped back to the target fixed-point format. Therefore, the same quantization module can support different scale parameters across projection stages and element-wise operations. In the current implementation, weights, scale parameters, and nonlinear lookup tables are initialized into on-chip PL memories through memory initialization files, while DDR is mainly used for runtime input and output data movement.

3.2 Linear Layers

As discussed in Section 2.2, the Slim Mamba algorithm can be mapped to three major hardware primitives. The first primitive is matrix-vector multiplication (MVM) implemented with multiply-accumulate (MAC) operations, which is mainly used for the input projection, the update-gate projection(dt projection), and the output projection. The second primitive is element-wise multiplication, or EWM, which is used for the weighted terms in the state update and for output gating. The third primitive is element-wise addition, or EWA, which is used for the state update and residual addition. In the SSM datapath, these three primitives correspond to the projection computation, the recurrent state update, and the output gating operation, respectively.

3.2.1 Reuse vs. Latency Trade-off

From a hardware design perspective, these primitives can be mapped with different resource-reuse strategies. One option is to build a unified reconfigurable PE array, where the processing elements switch among MAC, EWM, and EWA modes through a common mode signal. This approach maximizes the reuse of

multipliers, adders, and local buffers, thereby reducing area and resource cost. However, since different computation stages share the same array, this design may introduce mode-switching overhead, scheduling stalls, and a coarser pipeline granularity. Another option is to map the projection-heavy computations to a shared MAC fabric, while using lightweight vector EWM/EWA modules for state update and output gating. This design introduces some additional resources, but it allows projection, state update, and output gating to be connected in a finer-grained streaming pipeline.

Therefore, the current implementation adopts a compromise between resource reuse and pipeline granularity. The underlying PE array is capable of supporting MAC, EWM, and EWA modes through a unified mode signal. In the current Slim Mamba accelerator, however, the input projection, dt projection, and output projection paths mainly use the shared MAC fabric in MAC mode. The element-wise multiply/add operations in the SSM state update and the element-wise multiplication in output gating are implemented using dedicated vector modules. In other words, the design preserves the reconfigurable capability of MAC/EWM/EWA execution, but functionally separates projection computation from element-wise computation to achieve better pipeline granularity and module decoupling.

This choice is also reflected in the fine-grained scheduling of the SSM datapath. In a coarse-grained design, different stages tend to execute sequentially, which improves resource reuse but increases end-to-end waiting time. In contrast, fine-grained streaming allows each MAC tile to be forwarded immediately to the following state update and output gating stages. In our throughput analysis with $N = 64$ tiles, the fine-grained schedule reduces latency from

$$T_A = 29N \tag{3.1}$$

to

$$T_B = 22N + 7, \tag{3.2}$$

that is, from 1856 cycles to 1415 cycles, corresponding to a 23.8% reduction. This schedule adds only about 12 DSPs per SSM compute path, which is acceptable compared with the achieved latency reduction. Based on this trade-off, the design adopts a fine-grained FIFO-based AXI-stream control scheme instead of relying entirely on a single reconfigurable array for all computations.

3.2.2 MAC Array Architecture Selection

The core workload of the linear layers can be abstracted as matrix-vector multiplication. Taking the dt projection as an example, its linear part can be expressed as

$$a_t = W_\lambda u_t + b_\lambda, \tag{3.3}$$

where u_t is the state input, and W_λ and b_λ are the projection weight and bias, respectively. This computation is performed by the MAC fabric. Similarly, the input projection generates the state input and gate input from the input feature, while the output projection maps the gated result back to the block output dimension. These projections are all MAC operations and can therefore reuse the same MAC fabric with different input buffers, weight buffers, and output destinations.

Table 3.1: Comparison of candidate MAC array organizations for projection computation.

	16×16 Systolic	64×1 GEMV	$4 \times 4 \times 4$ Pipeline
Wires	+++	+	++
DSPs	256	64	64
BW (weights/cycle)	256	64	64
Clock cycles for $(256, 256)$ MVM	256	1024	1024

Take SSM projection workload for example, a typical linear layer can be modeled as the multiplication between a 256×256 matrix and a 256×1 vector, resulting in

$$256 \times 256 = 65,536 \tag{3.4}$$

MAC operations. Given a fixed number of processing elements, the ideal lower bound of the compute cycles can be estimated by

$$CC_{\min} = \left\lceil \frac{\text{Total MAC operations}}{\#\text{PEs}} \right\rceil. \tag{3.5}$$

This formula only gives a theoretical lower bound. The actual latency is also affected by pipeline fill and drain, reduction, scheduling, and data delivery. Based on this workload, three candidate MAC array organizations are compared in Table 3.1. Here, the number of "+" symbols qualitatively indicates the relative wiring complexity.

A large 16×16 systolic array provides high spatial parallelism and a low theoretical cycle count. However, for the projection workload in this design, the computation is dominated by GEMV rather than large-batch GEMM. Therefore, the two-dimensional data reuse advantage of a large systolic array is not fully exploited, while the larger PE scale introduces higher global routing, control, and data-supply pressure.

A 64×1 GEMV engine is more directly matched to matrix-vector multiplication. However, in a typical 64×1 GEMV engine, if all 64 MACs serve one output lane, 64 products are generated in one cycle and must be reduced into one partial sum for that output channel. This usually requires a 64-input adder tree, or a multi-stage pipelined adder tree. Although such a structure is suitable for pure GEMV computation, the large adder-tree fan-in may increase the number of adder stages, routing complexity, and the pressure of pipeline register insertion, making timing closure more challenging on FPGA.

The proposed $4 \times 4 \times 4$ MAC array organizes the 64 PEs as four cascaded 4×4 sub-arrays. Each sub-array processes a small input-weight tile, while the four sub-arrays are activated with a staggered schedule to combine local spatial parallelism and inter-array pipelining. The comparison above shows the advantage of this organization. Compared with a 64×1 GEMV engine, it provides higher vector-output parallelism and reduces the pressure of a large top-level reduction tree. Compared with a large 16×16 systolic array, it avoids excessive global routing and data-supply pressure for the MVM-dominated workload. Therefore, the $4 \times 4 \times 4$ structure is selected as a balanced shared MAC fabric in terms of resource cost, pipeline regularity, and timing closure.

Table 3.4: Weight input scheduling for the pipelined MAC arrays.

Cycle	Array1 Columns	Array2 Columns	Array3 Columns	Array4 Columns	Description
1	col0–3	–	–	–	ARRAY1 preloads first 4×4 block (tile1).
2	col16–19	col4–7	–	–	ARRAY2 begins tile1.
3	col32–35	col20–23	col8–11	–	ARRAY3 begins tile1 (3-cycle stagger).
4	col48–51	col36–39	col24–27	col12–15	ARRAY4 joins; pipeline full.
5–16	continue +16 stride	same	same	same	Steady-state loading of tile1.
17	**col256–259 → tile2**	col244–247	col232–235	col220–223	**ARRAY1 starts tile2 (row4–7).**
18	col272–275	**col260–263 → tile2**	col248–251	col236–239	**ARRAY2 switches to tile2.**
19	col288–291	col276–279	**col264–267 → tile2**	col252–255	**ARRAY3 switches to tile2.**
20	col304–307	col292–295	col280–283	**col268–271 → tile2**	**ARRAY4 switches; tile1 finishes.**
21–37	continue +16 stride	same	same	same	Steady-state operation for tile2.
38	**tile3 preload**	–	–	–	ARRAY1 starts tile3.
39	–	**tile3 preload**	–	–	ARRAY2 starts tile3.
40	–	–	**tile3 preload**	–	ARRAY3 starts tile3.
41	–	–	–	**tile3 preload**	ARRAY4 starts tile3 (3-cycle stagger).
42–58	continue +16 stride	same	same	same	Steady tile3 operation.

At each cycle, a sub-array receives a 4-element slice of the input vector together with a 4×4 weight block. The first sub-array starts a new output tile, and the following sub-arrays join the same tile with a one-cycle offset. Weights are scheduled in 4-column blocks. For each sub-array, the column start index advances by a fixed stride of 16 every cycle, as summarized in Table 3.2. Within the same cycle, the four sub-arrays access distinct column blocks with fixed offsets $\{0, 4, 8, 12\}$, resulting in a constant 12-column spacing between adjacent active arrays, as shown in Table 3.3.

Table 3.2: Column starts per sub-array.

Array	Column Start Points	Δ
Array1	$0 \rightarrow 16 \rightarrow 32 \rightarrow 48 \rightarrow 64$	+16
Array2	$4 \rightarrow 20 \rightarrow 36 \rightarrow 52$	+16
Array3	$8 \rightarrow 24 \rightarrow 40$	+16
Array4	$12 \rightarrow 28$	+16

Table 3.3: 12-column array spacing.

Cycle	A1→A2 Δ	A2→A3 Δ	A3→A4 Δ
2	16–4 = **12**	–	–
3	32–20 = **12**	20–8 = **12**	–
4	48–36 = **12**	36–24 = **12**	24–12 = **12**
5	64–52 = **12**	52–40 = **12**	40–28 = **12**

Also, the input vector x_t follows the same staggered pipeline as the weight tiles. Each sub-array receives the activation slice that matches its current 4×4 weight block, and these slices are delayed together with the corresponding weights to keep the four sub-arrays aligned. After the 16 column groups of the current output-row tile have been consumed, the scheduler switches to the next output-row tile. Table 3.4 illustrates this tile-switching pattern.

3.2.3 Hardware Implementation of Linear Layers

An example of the array hardware implementation is shown in Fig. 3.1(d). Local accumulation is first performed inside the 4×4 MAC arrays, and the partial results generated by the four sub-arrays are then sent to a 4-input reduction tree to form the vector output. The reduced vector does not need to wait for all later partial vectors of the layer to be completed. Instead, it is forwarded through FIFO interfaces to the following stage, such as the bias-add or requantization stage in the SSM projection datapath. These FIFOs decouple the reduction output from the downstream pipeline and keep the data transfer stable under valid/ready control.

Therefore, the $4 \times 4 \times 4$ structure uses small-granularity tiles, array cascade, cross-cycle accumulation, and FIFO-based streaming to distribute computation and reduction pressure into a more regular and localized structure.

3.3 Non-linear Layers

3.3.1 Normalization

In the Slim-Mamba block as shown in Fig. 2.1, the normalization layer is placed before the input projection to regulate the dynamic range of the input tokens. Since the subsequent linear projection, recurrent state update, and gating operations are sensitive to the scale of intermediate activations, large variations across channels or time steps may lead to numerical instability after fixed-point quantization. Therefore, normalization is introduced to rescale the input features into a more stable numerical range before they enter the main datapath.

Following the computation structure of Mamba-family blocks, where RMS normalization is commonly used as part of the block-level computation path [9, 10], this work adopts RMSNorm as the normalization layer in Slim-Mamba. For an input feature vector $\mathbf{x} = [x_0, x_1, \ldots, x_{D-1}]$, the RMS value is computed as

$$\mathrm{RMS}(\mathbf{x}) = \sqrt{\frac{1}{D}\sum_{i=0}^{D-1} x_i^2 + \epsilon}, \tag{3.6}$$

where D denotes the feature dimension and ϵ is a small positive constant used to avoid division by zero. The normalized output is then computed as

$$\hat{x}_i = \frac{x_i}{\mathrm{RMS}(\mathbf{x})}. \tag{3.7}$$

From a hardware perspective, RMSNorm can be decomposed into square accumulation, mean scaling, square-root computation, and division. The square accumulation is implemented using multipliers and an adder tree. The mean scaling corresponds to division by the feature dimension D; when D is a power of two, this operation can be implemented by a right shift. In contrast, the square-root and division operations are relatively expensive in a fixed-point FPGA datapath. Sutikno et al. investigated FPGA implementations of non-restoring square-root algorithms and showed that such digit-recurrence methods can be hardware friendly [12]. Motivated by this, our work adopts a trial-quotient square-root procedure which is consistent with digit-recurrence or non-restoring square-root algorithms. This algorithm is attractive for FPGA implementation because it relies on a regular sequence of shift, comparison, subtraction, and control operations, without requiring a general floating-point square-root unit, a large lookup table, or a large number of multipliers.

The concrete square-root computation is shown in Fig. 3.2. The input is the fixed-point value `mean_sq_q16` $+ \epsilon$, and both the partial remainder and the root register are initialized to zero. During each iteration, the next input bits are shifted into the partial remainder, and a trial value is generated from the current

root estimate. If the partial remainder is greater than or equal to the trial value, the trial subtraction is accepted and the current root bit is set to one. Otherwise, the remainder is kept unchanged and the current root bit is set to zero. After all root bits have been processed, the module outputs `rms_q88`, which is the fixed-point RMS value used by the subsequent normalization stage.

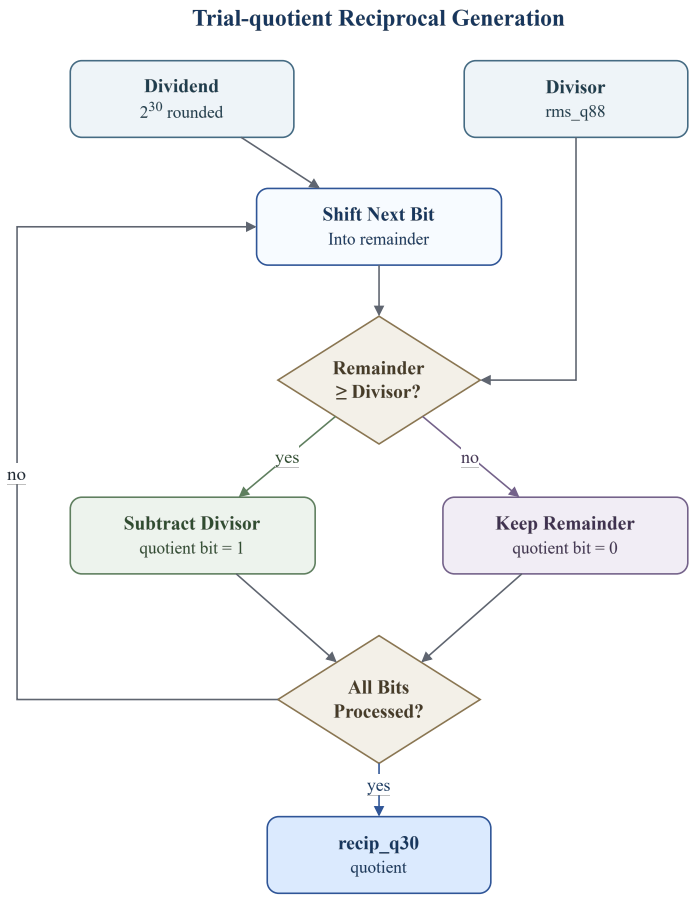

Figure 3.2: Trial-quotient square-root computation for the RMS value.

After `rms_q88` is obtained, the normalization still requires division by the RMS value. Directly instantiating a divider for each channel would lead to a high resource cost, especially because the same RMS value is shared by all channels of the current token. Therefore, the division in Eq. (3.7) is transformed into reciprocal approximation followed by multiplication:

$$\frac{x_i}{\mathrm{RMS}(\mathbf{x})} \approx x_i \cdot \frac{\widehat{1}}{\mathrm{RMS}(\mathbf{x})}. \tag{3.8}$$

The reciprocal-generation procedure is shown in Fig. 3.3. The module uses the

fixed-point constant 2^{30} as the dividend and `rms_q88` as the divisor. A quotient is generated bit by bit using a trial-quotient division process. At each step, the next dividend bit is shifted into the partial remainder. If the remainder is greater than or equal to the divisor, the divisor is subtracted and the current quotient bit is set to one; otherwise, the remainder is kept and the quotient bit is set to zero. After all quotient bits have been generated, the output `recip_q30` represents the Q30 fixed-point approximation of the RMS reciprocal. The normalized output is then produced by multiplying the input feature with `recip_q30`, followed by fixed-point shifting, rounding, and saturation.

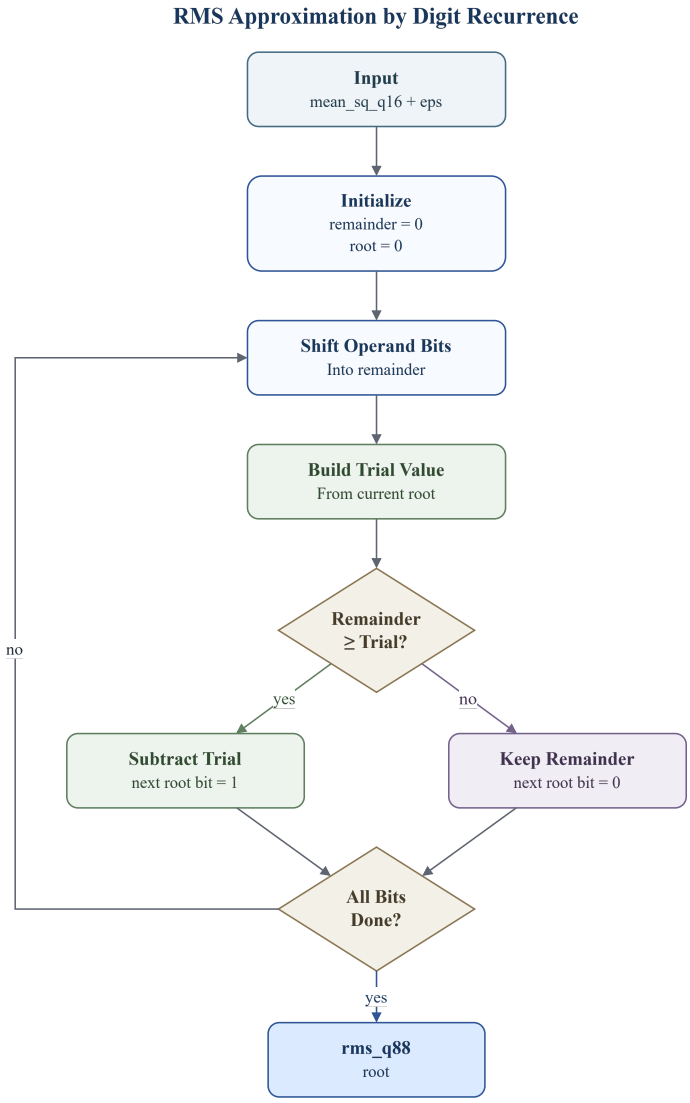

Figure 3.3: Reciprocal-based division approximation for RMSNorm.

A higher-precision Q30 reciprocal is used because the reciprocal acts as a shared scaling factor for all channels of the current token. If this reciprocal were represented with insufficient fractional precision, the same scaling error would be propagated to the entire normalized vector and further affect the following linear projection and state-update stages. Therefore, the reciprocal is computed with higher fractional precision and is only reduced to the target fixed-point format after multiplication, rounding, and saturation. Experimental validation shows that this precision is sufficient to preserve the numerical accuracy of the fixed-point normalization unit and the final localization performance.

The reciprocal-multiplication formulation converts the normalization division into one reciprocal generation followed by multiple multiplications. The reciprocal can be computed once and then reused across all channels. Istoan and Pasca compared fixed-point FPGA implementations of reciprocal, square-root, and reciprocal-square-root functions, and pointed out that these functions often share common implementation structures, with the objective of optimizing the use of fixed FPGA resources such as block multipliers and block RAMs[13]. Therefore, replacing direct division with reciprocal approximation and multiplication provides a hardware-friendly implementation for the proposed fixed-point RMSNorm datapath.

Overall, the proposed normalization unit preserves the function of RMSNorm while avoiding costly general-purpose square-root and division units. The division by the feature dimension is implemented by shifting, the square root is computed using a trial-quotient digit-recurrence procedure, and the final division is implemented through reciprocal approximation and multiplication. As shown in Fig. 3.4, this hardware approximation remains close to the floating-point RMSNorm reference, with a maximum absolute error of 0.015625 and a mean absolute error of 0.004211 in the validation case.

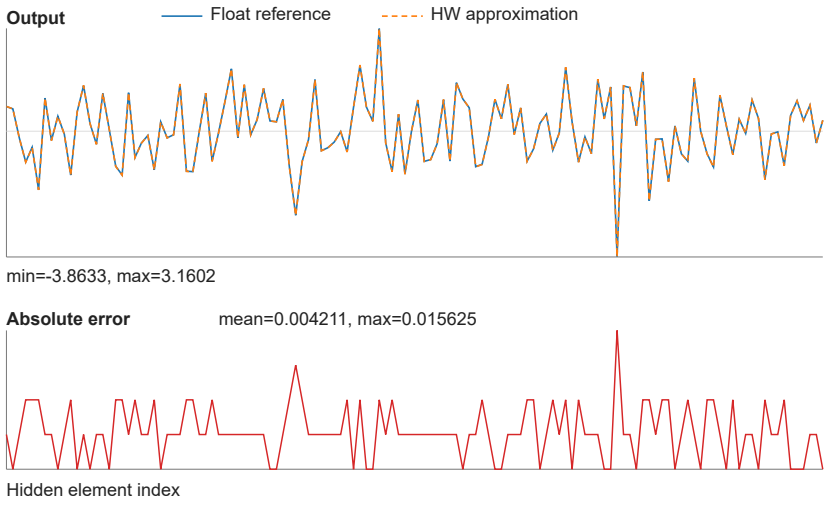

Figure 3.4: Comparison between floating-point RMSNorm and the fixed-point hardware approximation.

3.3.2 Sigmoid and SiLU

The sigmoid function is used in the SSM update path to generate bounded update coefficients. To realize the nonlinearity $\sigma(x) = 1/(1+e^{-x})$ with low FPGA overhead, this work implements a lookup-table (LUT) approximation. Before the lookup, the input is clamped to the interval $[-4, +4)$. This range is selected from empirical workload statistics: sampled dt_proj values before the sigmoid are concentrated in this interval, so the LUT does not need to cover large saturation regions with fine resolution.

After clamping, the LUT address is generated by shifting the selected input domain to a non-negative index range:

$$\text{addr} = x_{\text{clamp}} + x_{\text{offset}}. \tag{3.9}$$

Here, x_{offset} maps the lower bound of the clamped interval to address zero. A 2048-entry LUT is used to uniformly cover the effective sigmoid domain. Fig. 3.5 compares the floating-point sigmoid with the hardware LUT output, the error is below 0.02% showing that the chosen clamp range and table resolution provide a close approximation.

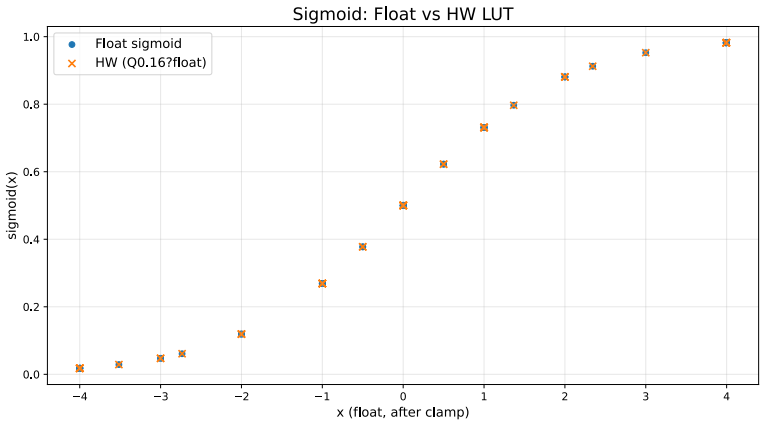

Figure 3.5: Comparison between floating-point sigmoid and the fixed-point LUT approximation.

The SiLU activation is implemented using the same sigmoid LUT path rather than a separate nonlinear unit. For an input x, SiLU is written as

$$\text{SiLU}(x) = x \cdot \sigma(x). \tag{3.10}$$

In hardware, the input vector is sent to the sigmoid LUT and, in parallel, delayed through a FIFO path. After the sigmoid value and the original input are aligned, an element-wise multiplier produces the SiLU output. This reuses the same clamp and LUT mechanism as the sigmoid unit, while adding only the alignment FIFO and vector multiplication required by the $x \cdot \sigma(x)$ formulation.

3.4 Quantization

3.4.1 Quantization Workflow

This subsection describes the quantization flow from the floating-point Slim-Mamba model to the fixed-point RTL implementation. In this design, quantization is not only a conversion of floating-point weights into integer values. The fixed-point rounding and saturation behavior in linear layers, nonlinear functions, state updates must also be considered. Therefore, the proposed flow uses three validation levels: a Python model, a C++ fixed-point model, and an RTL implementation.

The Python model is used as the golden reference. It preserves the floating-point Slim-Mamba behavior and allows fast evaluation of different quantization settings on the final localization accuracy. The C++ fixed-point model then reproduces the main integer operations used in hardware, including multiplication, shifting, lookup-table access, rounding, and saturation. This level is closer to RTL than the Python model and is used to define the bit-true behavior of each operator. Finally, the RTL implementation receives the quantized weights, bias terms, scale factors, nonlinear LUTs, and control parameters. Its output is compared against the C++ fixed-point reference.

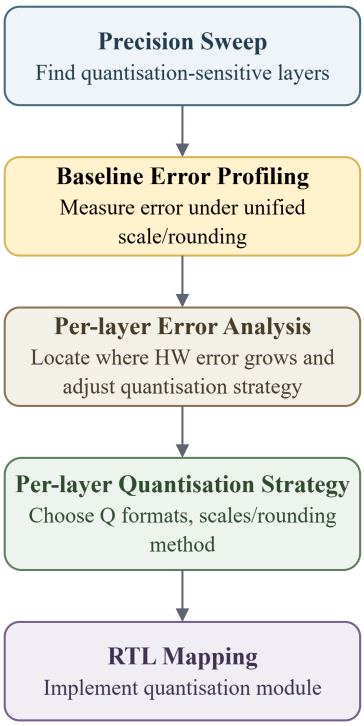

Figure 3.6: Quantization workflow from precision sweep to RTL mapping.

As shown in Fig. 3.6, the workflow starts with a precision sweep to determine the required precision for different computation stages. A baseline error profiling step is then performed using a unified projection requantization configuration. In this baseline, the input projection, *dt* projection, and output projection use identity Q15 scales, while the MAC results are written back using round-to-nearest ties-to-even and saturation clamp. For nonlinear layers, the sigmoid and SiLU path uses a Q0.16 sigmoid LUT output, while RMSNorm uses a Q30 reciprocal scale followed by fixed-point shifting, rounding, and saturation. If the final output error is too large, per-layer error analysis is used to locate where the mismatch grows. Based on this analysis, the layer-wise quantization strategy is refined and then mapped to the RTL quantization modules and memory initialization files.

The purpose of this workflow is to keep the quantization decisions traceable across algorithm evaluation and hardware execution. Python reflects the floating-point behavior of the model, C++ fixes the bit-true integer semantics, and RTL verifies the final hardware behavior. This step-by-step process helps identify the main quantization error sources before FPGA implementation and keeps the deployed fixed-point accelerator consistent with the earlier model validation.

3.4.2 Error Analysis and Layer-wise Quantization Strategy

Before the bit-true RTL mapping, a fake-quantization precision sweep is first used to estimate the accuracy impact of different bit-width choices. In this step, quantize and dequantize operations are inserted in the software model to emulate low-precision effects, while the evaluation is still performed before RTL implementation. As shown in Table 3.5, the all-int16 configuration achieves a localization error close to the fake-quantization reference. The sweep also shows that the `dt_proj`, `ssm_state`, and gate paths can be reduced to int8 with negligible loss, while `in_proj` and `out_proj` are more sensitive to precision reduction. However, the accelerator uses a shared reconfigurable MAC fabric for multiple projection stages. To keep the array control, datapath width, and on-chip memory organization simple, the final implementation adopts a unified 16-bit datapath instead of assigning different bit-widths to different stages.

Table 3.5: Fake-quantization precision sweep.

Configuration	Precision override	Mean localization error (m)
Fake-quant reference	Software fake-quant baseline	0.13695
All-int16	`ssm_state=int16`, `gate=int16`	0.13520
Mixed candidate	`dt_proj=int8`, `ssm_state=int8`, `gate=int8`	0.13519
`in_proj` int8	`in_proj=int8`, `ssm_state=int16`, `gate=int16`	0.76520
`out_proj` int8	`out_proj=int8`, `ssm_state=int16`, `gate=int16`	0.24352

After the layer precision is selected, a unified fixed-scale quantization scheme is first evaluated as the baseline. In this baseline, the output differs substantially from the floating-point output, with an MAE of 0.17987 and a mean localization error of 0.35543 m. To locate the error sources, the Slim-Mamba block is decomposed according to the dataflow in Fig. 3.7, and the error is measured after each computation stage. The corresponding mean absolute error (MAE) values are listed in Table 3.6. This stage-level analysis shows that the largest error increase

occurs in the selective scan stage. Since this stage repeatedly updates the recurrent state, its value range changes across channels and time steps, so a single fixed Q8.8 scale is not sufficient to represent both small and large state values. To reduce this error, the selective-scan state is changed to a dynamic-scale representation, where the scale is derived from the channel range. This reduces the selective-scan MAE from 0.127090 to 0.085484 and also improves the following gating, output projection, and block-output errors.

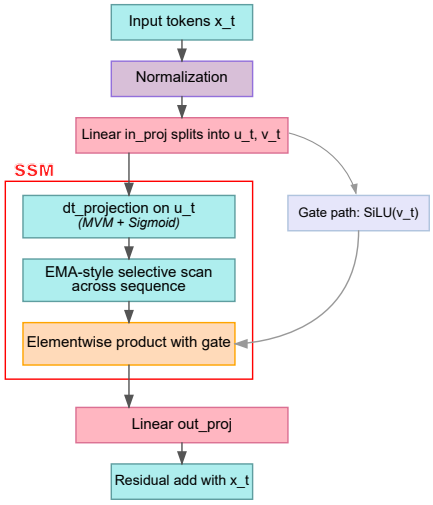

Figure 3.7: Slim-Mamba block flow used for stage-level quantization error analysis.

Table 3.6: Stage-level quantization error before and after dynamic scaling.

Stage	Fixed-scale MAE	Dynamic-scale MAE
`norm`	0.004498	0.004498
`inproj_u`	0.010917	0.010917
`inproj_z`	0.011069	0.011069
`silu_u`	0.006183	0.006183
`silu_gate`	0.005931	0.005931
`dtproj`	0.010139	0.010139
`dt_sigmoid`	0.002118	0.002118
`selective_scan`	0.127090	0.085484
`ewm_gating`	0.033001	0.029827
`outproj`	0.032225	0.031243

The improvement is also reflected in the final end-to-end evaluation in Table 3.7. The early fixed-scale baseline has a large output mismatch and a much higher localization error. After the dynamic-scale state representation is introduced, the hardware output MAE is reduced to 0.02081, and the final mean local-

ization error becomes 0.14103 m, which is close to the fake-quantization reference of 0.13695 m.

Table 3.7: End-to-end quantization evaluation.

Evaluation	Samples	HW-like vs. float MAE	Mean localization error (m)
fixed-scale	32	0.17987	0.35543
dynamic-scale	34,505	0.02081	0.14103
Fake-quant reference	34,505	–	0.13695

Based on the above analysis, the final layer-wise quantization strategy is summarized in Table 3.8. Most intermediate values use signed Q8.8 with a scale of 1/256. The sigmoid-related values use unsigned Q0.16, which matches the range of the sigmoid output. The selective-scan coefficient λ is represented as signed Q1.15, while the selective-scan state stored in SRAM uses a dynamic signed int16 scale. For rounding, the main rule is round-to-nearest-even after MAC operations, Q-format conversion, state update, and scale multiplication. Final Q8.8 values are clamped to the signed int16 range $[-32768, 32767]$, corresponding to approximately $[-128, 127.996]$, to prevent overflow or sign errors after multiplication, accumulation, and residual addition.

Table 3.8: Final layer-wise quantization strategy.

Scale	Q format	Layer or signal
1/256	Signed int16 Q8.8	RMSNorm output, `in_proj`, `dt_proj`, `out_proj`, `gate_y`
1/65536	Unsigned Q0.16	Sigmoid LUT output, SiLU sigmoid path, `dt_lambda`
1/32768	Signed Q1.15	Selective-scan λ
Dynamic scale	Scaled signed int16	Selective-scan state SRAM

3.4.3 Quantization Hardware Module Design

Instead of implementing a separate quantization unit for each layer, this work uses a shared configurable quantization engine. As shown in Fig. 3.8, the engine receives an input value, a selected scale, and several control modes. The input is first extended according to the input sign mode. It is then multiplied by either a fixed scale or a runtime dynamic scale. The scaled product is shifted to the target binary point, rounded according to the rounding mode, and finally converted to the target output range. The saturation mode selects whether the final value is clamped to the legal range or wrapped to the target bit width. This common structure is used after MAC accumulation, Q-format conversion, state update, scale multiplication, and residual addition. Compared with layer-specific quantization logic, the shared engine reduces duplicated RTL, keeps the rounding and saturation behavior consistent across the accelerator, and allows the scale, rounding, and saturation policies found in the software analysis to be mapped directly to hardware configuration parameters.

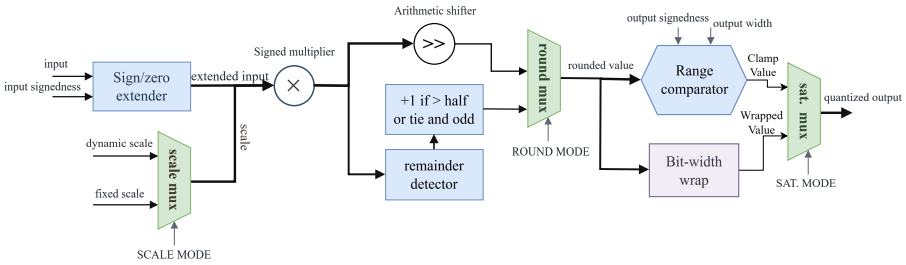

Figure 3.8: Configurable fixed-point quantization engine for one vector lane.

Dynamic scaling is used in the selective-scan state path, where the recurrent state is stored in SRAM and updated over time. The state range varies across channels and tokens, so a single fixed Q8.8 scale can either saturate large values or lose precision for small values. The runtime scale generator therefore derives scale factors from the current state input range and produces the scale values required for state storage and conversion back to Q8.8. Since the selective-scan datapath is stream based, the token, state address, update coefficient, and corresponding runtime scales must remain aligned. For this purpose, the design uses the ping-pong buffer structure shown in Fig. 3.9. While one bank receives the incoming token and its runtime scales, the other bank can provide an already aligned token-scale pair to the downstream state update unit. The write and read bank selectors alternate between the two banks, which hides the scale-generation latency and preserves the ordering of the streaming data. This mechanism allows dynamic scaling to be inserted into the state update path without breaking the fine-grained pipeline.

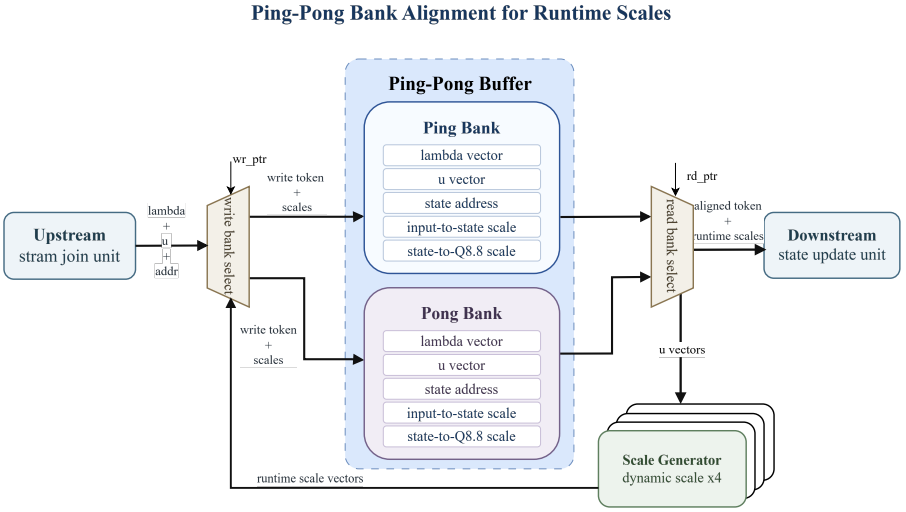

Figure 3.9: Ping-pong buffer alignment for runtime dynamic scales in the selective-scan path.

Chapter 4
Results and Discussion

4.1 Accuracy

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

4.2 Performance

Nulla malesuada porttitor diam. Donec felis erat, congue non, volutpat at, tincidunt tristique, libero. Vivamus viverra fermentum felis. Donec nonummy pellentesque ante. Phasellus adipiscing semper elit. Proin fermentum massa ac quam. Sed diam turpis, molestie vitae, placerat a, molestie nec, leo. Maecenas lacinia. Nam ipsum ligula, eleifend at, accumsan nec, suscipit a, ipsum. Morbi blandit ligula feugiat magna. Nunc eleifend consequat lorem. Sed lacinia nulla vitae enim. Pellentesque tincidunt purus vel magna. Integer non enim. Praesent euismod nunc eu purus. Donec bibendum quam in tellus. Nullam cursus pulvinar lectus. Donec et mi. Nam vulputate metus eu enim. Vestibulum pellentesque felis eu massa.

Quisque ullamcorper placerat ipsum. Cras nibh. Morbi vel justo vitae lacus tincidunt ultrices. Lorem ipsum dolor sit amet, consectetuer adipiscing elit. In hac habitasse platea dictumst. Integer tempus convallis augue. Etiam facilisis. Nunc elementum fermentum wisi. Aenean placerat. Ut imperdiet, enim sed gravida sollicitudin, felis odio placerat quam, ac pulvinar elit purus eget enim. Nunc vitae tortor. Proin tempus nibh sit amet nisl. Vivamus quis tortor vitae risus porta vehicula.

4.3 FPGA On-board Validation

Morbi luctus, wisi viverra faucibus pretium, nibh est placerat odio, nec commodo wisi enim eget quam. Quisque libero justo, consectetuer a, feugiat vitae, porttitor eu, libero. Suspendisse sed mauris vitae elit sollicitudin malesuada. Maecenas ultricies eros sit amet ante. Ut venenatis velit. Maecenas sed mi eget dui varius euismod. Phasellus aliquet volutpat odio. Vestibulum ante ipsum primis in faucibus orci luctus et ultrices posuere cubilia Curae; Pellentesque sit amet pede ac sem eleifend consectetuer. Nullam elementum, urna vel imperdiet sodales, elit ipsum pharetra ligula, ac pretium ante justo a nulla. Curabitur tristique arcu eu metus. Vestibulum lectus. Proin mauris. Proin eu nunc eu urna hendrerit faucibus. Aliquam auctor, pede consequat laoreet varius, eros tellus scelerisque quam, pellentesque hendrerit ipsum dolor sed augue. Nulla nec lacus.

Suspendisse vitae elit. Aliquam arcu neque, ornare in, ullamcorper quis, commodo eu, libero. Fusce sagittis erat at erat tristique mollis. Maecenas sapien libero, molestie et, lobortis in, sodales eget, dui. Morbi ultrices rutrum lorem. Nam elementum ullamcorper leo. Morbi dui. Aliquam sagittis. Nunc placerat. Pellentesque tristique sodales est. Maecenas imperdiet lacinia velit. Cras non urna. Morbi eros pede, suscipit ac, varius vel, egestas non, eros. Praesent malesuada, diam id pretium elementum, eros sem dictum tortor, vel consectetuer odio sem sed wisi.

4.4 Discussion

Suspendisse vel felis. Ut lorem lorem, interdum eu, tincidunt sit amet, laoreet vitae, arcu. Aenean faucibus pede eu ante. Praesent enim elit, rutrum at, molestie non, nonummy vel, nisl. Ut lectus eros, malesuada sit amet, fermentum eu, sodales cursus, magna. Donec eu purus. Quisque vehicula, urna sed ultricies auctor, pede lorem egestas dui, et convallis elit erat sed nulla. Donec luctus. Curabitur et nunc. Aliquam dolor odio, commodo pretium, ultricies non, pharetra in, velit. Integer arcu est, nonummy in, fermentum faucibus, egestas vel, odio.

Chapter 5
Conclusion

5.1 Conclusion

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

5.2 Future Work

Nam dui ligula, fringilla a, euismod sodales, sollicitudin vel, wisi. Morbi auctor lorem non justo. Nam lacus libero, pretium at, lobortis vitae, ultricies et, tellus. Donec aliquet, tortor sed accumsan bibendum, erat ligula aliquet magna, vitae ornare odio metus a mi. Morbi ac orci et nisl hendrerit mollis. Suspendisse ut massa. Cras nec ante. Pellentesque a nulla. Cum sociis natoque penatibus et magnis dis parturient montes, nascetur ridiculus mus. Aliquam tincidunt urna. Nulla ullamcorper vestibulum turpis. Pellentesque cursus luctus mauris.

Chapter 6
Comments on LaTeX references

If you want to know more about LaTeX there is a (free) manual at [14]. For more specific questions, it is recommended to have a look at the forum StackExchange [15], where the most common questions already have answers. All official packets can be found, and downloaded, from CTAN [16]. Finally, for the hardcore programmer who thinks LaTeX is a bit inflexible, I can recommend the TeX introduction in [17].

For questions about how you should do to get your imported graphics as you want, have a look at [18]. If you instead want to do the images inline from the TeX code you are recommended to use TikZ [19]. However, it is known to have a relatively high learning threshold.

References

[1] A. Bourdoux, A. N. Barreto, B. Van Liempd, et al., "6G white paper on localization and sensing," arXiv preprint arXiv:2006.01779, 2020.

[2] S. E. Trevlakis, A. A. A. Boulogeorgos, D. Pliatsios, et al., "Localization as a key enabler of 6G wireless systems: A comprehensive survey and an outlook," *IEEE Open Journal of the Communications Society*, 2023.

[3] F. Mogyorósi, P. Revisnyei, A. Pašić, et al., "Positioning in 5G and 6G networks—A survey," *Sensors*, vol. 22, no. 13, p. 4757, 2022.

[4] G. Tian, I. Yaman, M. Sandra, et al., "Deep-learning-based high-precision localization with Massive MIMO," *IEEE Transactions on Machine Learning in Communications and Networking*, vol. 2, pp. 19–34, 2024.

[5] G. Tian, I. Yaman, M. Sandra, et al., "High-precision machine-learning based indoor localization with Massive MIMO system," in *ICC 2023 — IEEE International Conference on Communications*, Rome, Italy, 2023, pp. 3690–3695.

[6] A. S. Yaro, F. Maly, and P. Prazak, "A survey of the performance-limiting factors of a 2-dimensional RSS fingerprinting-based indoor wireless localization system," *Sensors*, vol. 23, no. 5, p. 2545, 2023.

[7] A. Gu and T. Dao, "Mamba: Linear-time sequence modeling with selective state spaces," arXiv preprint arXiv:2312.00752, 2023.

[8] C. Zeng, Z. Liu, G. Zheng, and L. Kong, "CMamba: Channel correlation enhanced state space models for multivariate time series forecasting," arXiv preprint arXiv:2406.05316, 2024.

[9] R. Wei, S. Xu, L. Zhong, et al., "LightMamba: Efficient Mamba acceleration on FPGA with quantization and hardware co-design," in *2025 Design, Automation & Test in Europe Conference (DATE)*, IEEE, 2025. Available: `https://arxiv.org/abs/2502.15260`.

[10] A. Wang, H. Shao, S. Ma, et al., "FastMamba: A high-speed and efficient Mamba accelerator on FPGA with accurate quantization," arXiv preprint arXiv:2505.18975, 2025. Available: `https://arxiv.org/abs/2505.18975`.

[11] X. Jin, H. Zheng, M. Nie, et al., "HCSAs: Hybrid computing systolic arrays for accelerating Mamba models with unified state space buffers and energy-efficient dataflow," in *Great Lakes Symposium on VLSI 2025*, New York: ACM, 2025, pp. 457–462, doi: 10.1145/3716368.3735187.

[12] T. Sutikno, A. Z. Jidin, A. Jidin, and N. R. N. Idris, "Strategies for FPGA implementation of non-restoring square root algorithm," *International Journal of Electrical & Computer Engineering*, 2014.

[13] M. Istoan and B. Pasca, "Fixed-point implementations of the reciprocal, square root and reciprocal square root functions," 2015. Available: `https://hal.science/hal-01229538`.

[14] T. Oetiker, H. Partl, I. Hyna, and E. Schlegl, *A (Not So) Short Introduction to LaTeX2e*.

[15] TeX StackExchange.

[16] The Comprehensive TeX Archive Network.

[17] P. Abrahams, K. Hargreaves, and K. Berry, *TeX for the Impatient*.

[18] K. Reckdahl, *Using Imported Graphics in LaTeX and pdfLaTeX*.

[19] T. Tantau, *The TikZ and PGF Packages*.

Appendix A

Some extra material

Lorem ipsum dolor sit amet, consectetuer adipiscing elit. Ut purus elit, vestibulum ut, placerat ac, adipiscing vitae, felis. Curabitur dictum gravida mauris. Nam arcu libero, nonummy eget, consectetuer id, vulputate a, magna. Donec vehicula augue eu neque. Pellentesque habitant morbi tristique senectus et netus et malesuada fames ac turpis egestas. Mauris ut leo. Cras viverra metus rhoncus sem. Nulla et lectus vestibulum urna fringilla ultrices. Phasellus eu tellus sit amet tortor gravida placerat. Integer sapien est, iaculis in, pretium quis, viverra ac, nunc. Praesent eget sem vel leo ultrices bibendum. Aenean faucibus. Morbi dolor nulla, malesuada eu, pulvinar at, mollis ac, nulla. Curabitur auctor semper nulla. Donec varius orci eget risus. Duis nibh mi, congue eu, accumsan eleifend, sagittis quis, diam. Duis eget orci sit amet orci dignissim rutrum.

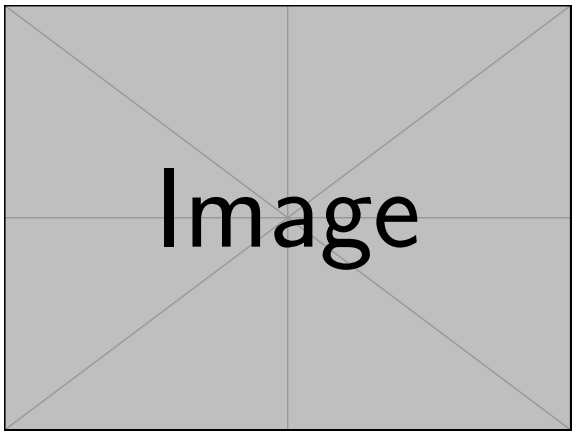

Figure A.1: A picture or table in the appendix is numbered accordingly.